Learning Geometric Algebra

Through Transformer Geometry

Agus Sudjianto

Sandi Setiawan

Aijun Zhang

A self-guided introduction to Geometric Algebra
with language models as your laboratory

Preface

What this book is

This is a book about Geometric Algebra. It teaches you a powerful mathematical framework — the Clifford algebra $\mathrm{Cl}(k, 0)$ — through a concrete, fascinating application: understanding how large language models process information internally.

You will learn GA one concept at a time. Each chapter introduces a single algebraic idea, develops it with small examples you can work by hand, and then immediately applies it to real data extracted from a transformer language model. The transformer is your *laboratory*: a place where abstract algebraic concepts become visible, measurable, and surprising.

Why transformers?

Geometric Algebra was born in physics and has thrived in computer graphics, robotics, and signal processing. But transformers offer something these fields do not: a high-dimensional space ($k = 256$ after whitening) where *every* GA concept — bivectors, rotors, holonomy, commutators — appears naturally in the data. In 3D physics, a bivector is just a dressed-up cross product. In $\mathbb{R}^{256}$, bivectors have 32,640 independent components, multiple principal planes, and rich internal structure. This is where GA earns its keep.

Moreover, transformers are *important*. Understanding their internal computation is one of the central challenges in AI. By learning GA through this lens, you gain two things at once: a deep understanding of a beautiful algebra, and a new way to see inside the models that are reshaping technology.

What you need

- **Linear algebra**: matrix multiplication, eigenvalues, orthogonal matrices, SVD. A first undergraduate course is sufficient.
- **Python**: NumPy-level comfort. The code is simple and self-contained.
- **Calculus**: partial derivatives for the dependency chapter. You can skip that chapter if needed.
- **No prior GA knowledge.** We start from vectors.
- **No prior ML knowledge.** Appendix A covers everything you need about transformers.

How to read this book

Each chapter follows the same pattern:

1. **The GA concept** — introduced abstractly with small ($\mathbb{R}^3$ or $\mathbb{R}^4$) examples.
2. **In the transformer** — the same concept applied to real hidden-state data from a language model.
3. **Code** — working Python using the `layer_time_ga` library. Every code block runs in the companion Jupyter notebooks.
4. **Exercises** — both pure GA problems and transformer-data explorations.

The chapters build sequentially: each depends on the previous. Read them in order the first time. After that, use the chapters as reference.

The companion notebooks

Each chapter has a companion Jupyter notebook in `tutorials_ga_learning/`. The notebooks contain the same code as the book, plus visualizations and interactive explorations. Run them alongside the book — seeing the algebra in data is the fastest way to build intuition.

The software

All code uses the `layer_time_ga` Python package, which wraps a standard PyTorch/NumPy backend with GA semantics. The student API `ltg_ga` provides a simple interface:

Listing 1: Three lines to GA

```python
import ltg_ga
model = ltg_ga.load_model("Qwen/Qwen2.5-7B")
result = ltg_ga.analyse("The capital of France is", model=model)
result.summary()
```

— *The Authors*

Contents

Preface **i**

I Vectors and Products **1**

1 Vectors Live Somewhere **3**
1.1 Vectors as geometric objects . . . 3
1.2 Our laboratory: transformer hidden states . . . 4
1.3 The inner product in hidden-state space . . . 4
1.4 What the dot product misses . . . 5

2 The Product That Does Everything **7**
2.1 The geometric product . . . 7
2.2 The outer product: an oriented plane . . . 8
2.3 In matrix form . . . 8
2.4 Hands-on: computing geometric products . . . 8
2.5 Grade decomposition . . . 9

3 Planes, Not Axes **11**
3.1 What is a bivector? . . . 11
3.2 Representation as skew-symmetric matrices . . . 12
3.3 Principal planes . . . 12
3.4 Bivector arithmetic . . . 12

4 When Your Coordinates Lie **15**
4.1 Frames and metrics . . . 15
4.2 What goes wrong without whitening . . . 15
4.3 Whitening as frame construction . . . 16
4.4 Choosing k: the dimensionality . . . 16

II Rotations and Rotors **19**

5 Rotations Without Matrices **21**

5.1 From matrices to rotors . . . 21
5.2 The sandwich product . . . 21
5.3 Constructing rotors from orthogonal matrices . . . 22
5.4 Rotor composition . . . 22

6 What a Layer Actually Does **25**
6.1 The versor decomposition . . . 25
6.2 Extracting the rotor field . . . 25
6.3 What the rotor field tells you . . . 26
6.4 Grade-0 vs grade-2: which dominates? . . . 26

7 Reading the Planes **29**
7.1 Principal plane extraction . . . 29
7.2 Plane evolution across layers . . . 30
7.3 Plane similarity between layers . . . 30

8 The Eigenvalue Story **31**
8.1 Eigenvalue spectra . . . 31
8.2 Why grade-0 dominates . . . 31
8.3 Reading the spectrum . . . 32

III Curvature and Commutators **33**

9 When Order Matters **35**
9.1 The commutator as a bivector . . . 35
9.2 Computing commutators . . . 35
9.3 The commutator field . . . 36

10 Walking in Circles **37**
10.1 Parallel transport on the grid . . . 37
10.2 The holonomy rotor . . . 37
10.3 Computing holonomy . . . 38
10.4 Curvature patterns in transformers . . . 38

11 How Much Computation? **39**
11.1 Accumulated non-commutativity . . . 39
11.2 Dependency weighting . . . 39
11.3 Plane decomposition of capacity . . . 40

IV The Full Picture **41**

12 Which Planes Matter? **43**
12.1 Dependency density . . . 43

12.2 Effective work: dependency × rotation . . . 43
12.3 Dependency-weighted bivectors . . . 44

13 Diagnosing and Steering **45**
13.1 Application 1: Does the model use context? . . . 45
13.2 Application 2: Hallucination signatures . . . 46
13.3 Application 3: Plane-specific steering . . . 46

14 What Geometric Algebra Gave Us **47**
14.1 The journey . . . 47
14.2 What was invisible before . . . 48
14.3 Where to go next . . . 49
14.4 Further reading . . . 49

A Transformers: A Primer **51**
A.1 The architecture . . . 51
A.2 Autoregressive prediction . . . 51
A.3 The models we study . . . 51

B GA from Scratch **53**
B.1 The Clifford algebra $\mathrm{Cl}(k, 0)$ . . . 53
B.2 Blades and multivectors . . . 53
B.3 The spin group . . . 53
B.4 The Lie algebra . . . 54
B.5 Duality and the pseudoscalar . . . 54

C The Matrix–Multivector Bridge **55**

D Code Reference **57**
D.1 Student API: `ltg_ga` . . . 57
D.2 Core algebra: `layer_time_ga.algebra` . . . 57
D.3 Decomposition: `layer_time_ga.decomposition` . . . 57
D.4 Curvature: `layer`$_t$*ime_ga.curvature* . . . 58
D.5 Capacity: `layer_time_ga.capacity` . . . 58

Glossary **60**

Part I

Vectors and Products

CHAPTER 1

Vectors Live Somewhere

Why This Matters

Before we can build Geometric Algebra, we need to think carefully about what a vector *is* — not as a column of numbers, but as an element of a space with geometric structure. This chapter establishes the foundation: vectors, inner products, and the spaces they live in. We then meet our laboratory: the hidden states of a transformer, which are vectors in a very high-dimensional space.

1.1 Vectors as geometric objects

You have been computing with vectors since your first linear algebra course: adding them, scaling them, taking dot products. But consider what the dot product actually *does*. For vectors $a, b \in \mathbb{R}^k$:

$$a \cdot b = \sum_{i=1}^{k} a_i b_i = \|a\| \, \|b\| \cos\theta$$

This single operation gives you three things:

1. **Length**: $\|a\| = \sqrt{a \cdot a}$.
2. **Angle**: $\cos\theta = a \cdot b / (\|a\| \|b\|)$.
3. **Projection**: the component of b along a.

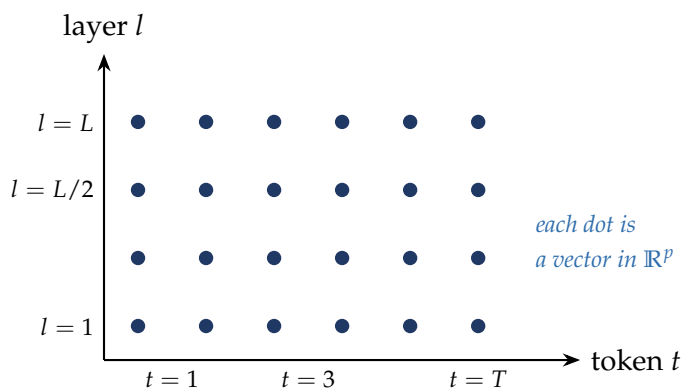
These are *geometric* properties. They do not depend on which coordinate system you use — only on the relationship between the vectors. The dot product is the beginning of geometry.

GA Concept

Grade 0: Scalars. The dot product takes two vectors (grade-1 objects) and produces a scalar (grade-0 object). This is our first example of *grade reduction*: combining elements of one grade to produce elements of a lower grade. We will see the opposite — grade *increase* — in the next chapter.

1.2 Our laboratory: transformer hidden states

A transformer language model with L layers processing T tokens holds a hidden-state vector $H(l,t) \in \mathbb{R}^p$ at every point of a two-dimensional grid:



For the Qwen-2.5-7B model: $L = 28$ layers, T depends on the prompt length, and $p = 3584$ dimensions per hidden state. That is $28 \times T$ vectors, each with 3584 components.

In the Transformer

When you type "The capital of France is" into a language model, the model creates $T = 6$ token representations. Each passes through 28 layers. At each step, the hidden state $H(l,t)$ changes — rotated, stretched, mixed with other tokens. The model's "thinking" is encoded in how these vectors change across the grid.

1.3 The inner product in hidden-state space

We can take the dot product of any two hidden states:

$$H(l_1,t_1) \cdot H(l_2,t_2) = \sum_{i=1}^{p} H_i(l_1,t_1)\, H_i(l_2,t_2)$$

This tells us how *aligned* two hidden states are. Same layer, adjacent tokens? Tells you if the model represents those tokens similarly. Same token, adjacent layers? Tells you how much the representation changed.

But there is a problem: the raw hidden-state space is *distorted*. Some dimensions have enormous variance while others are nearly dead. The dot product in the raw space is dominated by the high-variance dimensions and misses structure in the low-variance ones. We will fix this in Chapter 4 with whitening.

Listing 1.1: Extracting hidden states and measuring inner products

```python
import ltg_ga
import numpy as np

model = ltg_ga.load_model("Qwen/Qwen2.5-7B")
result = ltg_ga.analyse("The capital of France is", model=model)

H = result.H_whitened   # shape (L, T, k)
L, T, k = H.shape
print(f"Grid: {L} layers x {T} tokens x {k} dims")

# Inner product between adjacent tokens at layer 10
a = H[10, 0]  # first token, layer 10
b = H[10, 1]  # second token, layer 10
cos_angle = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print(f"Cosine similarity between tokens 0 and 1: {cos_angle:.4f}")
```

1.4 What the dot product misses

The dot product tells you *how much* two vectors overlap. It does not tell you *how* they differ. If $a \cdot b = 0$ (perpendicular), you know they point in different directions — but which directions? What is the *plane* they span?

This is the question that Geometric Algebra answers. The dot product is grade 0 (a scalar). The information it discards — the plane, the orientation — is grade 2 (a bivector). In the next chapter, we will see how the **geometric product** captures both at once.

Try It Yourself

Notebook: Chapter 1. Compute the cosine similarity matrix between all token pairs at a fixed layer. Plot it as a heatmap. Then compare heatmaps from an early layer ($l = 2$) and a late layer ($l = 26$). Tokens become more similar in late layers as the model converges toward a prediction.

CHAPTER 2

The Product That Does Everything

Why This Matters

Standard linear algebra has two products for vectors: the dot product ($a \cdot b$, a scalar) and the cross product ($a \times b$, a vector in $\mathbb{R}^3$ only). Geometric Algebra replaces both with a single operation — the **geometric product** — that works in any dimension, captures both angle and plane, and produces a richer algebraic object. This is the foundational operation of the entire framework.

2.1 The geometric product

For two vectors $a, b \in \mathbb{R}^k$, the geometric product is:

$$\boxed{a\,b = a \cdot b + a \wedge b}$$

It has two parts:

- $a \cdot b = \sum_i a_i b_i$ is the **inner product** (grade 0, a scalar). You already know this.
- $a \wedge b$ is the **outer product** (grade 2, a bivector). This is new.

The geometric product does not discard information. It contains both the "how much" (inner product) and the "in which plane" (outer product).

GA Concept

The geometric product is the fundamental operation of GA. Everything else — bivectors, rotors, holonomy — is built from it. In standard linear algebra, the inner product and the cross product are separate operations with different types. In GA, they are the grade-0 and grade-2 parts of a single product. This unification is what makes GA powerful.

2.2 The outer product: an oriented plane

The outer product $a \wedge b$ represents the **oriented plane** spanned by a and b. Its magnitude is the area of the parallelogram:

$$\|a \wedge b\| = \|a\| \, \|b\| \sin\theta$$

Compare this to the inner product: $a \cdot b = \|a\|\|b\| \cos\theta$. Together, they capture the full relationship:

Relationship	$a \cdot b$	$\|a \wedge b\|$
Parallel ($\theta = 0$)	maximal	0
Perpendicular ($\theta = \pi/2$)	0	maximal
45 degrees	equal	equal

Analogy

Think of two arrows lying on a table. The inner product tells you how much one arrow's shadow falls along the other. The outer product tells you the parallelogram they form — its area and which way it faces (clockwise or counterclockwise). The geometric product gives you *both*: shadow and parallelogram, in one object.

2.3 In matrix form

The outer product can be represented as a $k \times k$ **skew-symmetric matrix**:

$$(a \wedge b)_{ij} = a_i b_j - a_j b_i$$

This matrix satisfies $B^\top = -B$ (skew-symmetric). It has $k(k-1)/2$ independent entries — exactly the number of independent planes in $\mathbb{R}^k$.

For $k = 3$: 3 planes (xy, xz, yz) — same as the 3 components of the cross product. For $k = 256$ (our whitened space): 32,640 planes.

2.4 Hands-on: computing geometric products

Listing 2.1: The geometric product of two vectors

```python
from layer_time_ga.algebra import geometric_product_vectors
import numpy as np

# Perpendicular vectors in R^4
a = np.array([1.0, 0.0, 0.0, 0.0])
b = np.array([0.0, 1.0, 0.0, 0.0])

gp = geometric_product_vectors(a, b)
```

```python
print(f"Scalar (inner product): {gp['scalar']:.4f}")  # 0
print(f"Bivector norm (outer product): {gp['bivector'].norm:.4f}")  # 1

# Parallel vectors
c = np.array([3.0, 0.0, 0.0, 0.0])
gp2 = geometric_product_vectors(a, c)
print(f"Scalar: {gp2['scalar']:.4f}")  # 3
print(f"Bivector norm: {gp2['bivector'].norm:.6f}")  # ~0
```

In the Transformer

Take two hidden states from the same layer, different tokens. Their geometric product has a scalar part (how aligned are the token representations?) and a bivector part (what plane separates them?). If the scalar is large and the bivector small, the tokens are represented similarly. If the bivector is large, they live in different parts of the representation space, and the bivector tells you *which plane* separates them.

2.5 Grade decomposition

Any square matrix M can be decomposed into its grade-0 and grade-2 parts:

$$M = \underbrace{\frac{M + M^\top}{2}}_{\text{grade-0 (symmetric)}} + \underbrace{\frac{M - M^\top}{2}}_{\text{grade-2 (skew-symmetric)}}$$

This is the simplest possible grade decomposition. The symmetric part contains scalar/stretching information; the skew-symmetric part contains rotation/plane information.

Listing 2.2: Grade decomposition of a matrix

```python
from layer_time_ga.algebra import grade_decomposition

M = np.random.randn(8, 8)
gd = grade_decomposition(M)

print(f"Grade-0 norm: {gd['grade_0_norm']:.4f}")
print(f"Grade-2 norm: {gd['grade_2_norm']:.4f}")
print(f"Reconstruction error: "
      f"{np.linalg.norm(M - gd['grade_0'] - gd['grade_2'].matrix):.2e}")
```

Try It Yourself

Notebook: Chapter 2.

1. Compute the geometric product of two random unit vectors in $\mathbb{R}^8$. Verify that the scalar part is the dot product and the bivector is skew-symmetric.
2. Find two vectors where the geometric product is pure scalar (no bivector). What is their geometric relationship?
3. Find two vectors where the geometric product is pure bivector (no scalar). What does this mean?

CHAPTER 3

Planes, Not Axes

Why This Matters

Bivectors are the heart of GA. They represent oriented planes — the natural setting for rotations. In $\mathbb{R}^3$, you can get away with representing rotations by axes (vectors perpendicular to the plane). In higher dimensions, there is no unique "axis" — a rotation in $\mathbb{R}^{256}$ acts in a 2D plane within a 256D space. The bivector is the only object that correctly represents this.

3.1 What is a bivector?

A **bivector** B is a grade-2 element of the Clifford algebra. It represents an oriented plane:

- **Direction**: which 2D subspace (plane) of $\mathbb{R}^k$.
- **Magnitude**: the "area" of the bivector.
- **Orientation**: clockwise vs. counterclockwise.

The simplest bivector is $a \wedge b$ for two vectors a, b. This is called a **simple bivector** (or **blade**). It represents exactly one plane.

In higher dimensions, a *general* bivector is a sum of simple bivectors:

$$B = \sum_j B_j = \sum_j a_j \wedge b_j$$

This represents activity in *multiple* planes simultaneously.

GA Concept

Simple vs. compound bivectors.

- A **simple bivector** represents a single plane of rotation. Eigendecomposition of its skew matrix gives one conjugate pair $\pm i\sigma$.
- A **compound bivector** represents rotation in multiple planes. Its skew matrix has multiple

conjugate pairs $\pm i\sigma_1, \pm i\sigma_2, \ldots$.

In a transformer, each layer's bivector is typically compound: the layer rotates in several planes simultaneously. The **principal planes** (ordered by σ_j) reveal which planes carry the most rotation.

3.2 Representation as skew-symmetric matrices

A bivector in $\mathbb{R}^k$ is represented by a $k \times k$ skew-symmetric matrix:

$$B \in \mathbb{R}^{k \times k}, \qquad B^\top = -B$$

This has $k(k-1)/2$ independent entries. The basis bivectors $e_i \wedge e_j$ (for $i < j$) span this space.

Listing 3.1: Creating and inspecting a bivector

```
from layer_time_ga.algebra import bivector_from_skew, Bivector

# Create a random skew-symmetric matrix
rng = np.random.default_rng(42)
A = rng.standard_normal((8, 8))
A = 0.5 * (A - A.T)  # make skew-symmetric

B = bivector_from_skew(A)
print(f"Dimension: {B.dim}")
print(f"Independent components: {B.n_components}")
print(f"Norm: {B.norm:.4f}")
```

3.3 Principal planes

A compound bivector can be decomposed into its **principal planes** via eigendecomposition of the skew-symmetric matrix. The eigenvalues come in conjugate pairs $\pm i\sigma_j$, each corresponding to a simple bivector (one plane of rotation with angle σ_j).

Listing 3.2: Extracting principal planes

```
planes = B.principal_planes(n_planes=3)
for i, p in enumerate(planes):
    print(f"Plane {i}: angle = {p['angle']:.4f}, "
          f"weight = {p['weight']:.4f}")
```

The weights tell you how the rotation is distributed across planes. If the top plane has most of the weight, the rotation is nearly simple. If the weights are spread, the rotation acts in many planes.

3.4 Bivector arithmetic

Bivectors form a vector space: you can add, subtract, and scale them.

$$B_1 + B_2 \quad \text{(sum of two rotation planes)}$$

$$\alpha B \quad \text{(scale the rotation magnitude)}$$

This lets you do things like compute the *average* bivector across layers, or the *difference* between bivectors from two prompts.

In the Transformer

Each layer transition in a transformer has a bivector $\mathcal{B}^{(l)}$ that encodes its rotation planes. By extracting the principal planes, we can answer: does layer 5 rotate in the same plane as layer 25? Do arithmetic prompts and reasoning prompts produce different rotation planes? These questions are unanswerable with scalar summaries like $\|U - I\|_F$ — they require the bivector.

Try It Yourself

Notebook: Chapter 3.

1. Create a simple bivector from two orthonormal vectors in $\mathbb{R}^8$. Verify it has exactly one principal plane.
2. Create a compound bivector by adding two simple bivectors in orthogonal planes. Verify it has two principal planes.
3. Extract the bivectors from a real transformer analysis. How many significant principal planes does each layer's bivector have?

CHAPTER 4

When Your Coordinates Lie

Why This Matters

Geometric Algebra assumes you are working in an orthonormal frame: $e_i \cdot e_j = \delta_{ij}$. Raw transformer hidden states violate this assumption badly. This chapter teaches you why the frame matters, what goes wrong without it, and how whitening constructs the orthonormal frame that makes all subsequent GA operations valid.

4.1 Frames and metrics

A **frame** is a set of basis vectors $\{e_1, \ldots, e_k\}$ for your space. The **metric tensor** $g_{ij} = e_i \cdot e_j$ tells you how the basis vectors relate to each other.

In an **orthonormal frame**: $g_{ij} = \delta_{ij}$ (the identity matrix). This means:

- The basis vectors are mutually perpendicular.
- Each basis vector has unit length.
- The dot product $a \cdot b = \sum_i a_i b_i$ is the "true" geometric inner product — no correction needed.

GA Concept

The Clifford algebra $\mathrm{Cl}(k, 0)$**.**

Our algebra is defined over $\mathbb{R}^k$ with a positive-definite metric: $e_i^2 = e_i \cdot e_i = +1$ for all i. This is denoted $\mathrm{Cl}(k, 0)$ (the second number, 0, means no negative-signature directions). For this algebra to work correctly, we need coordinates in which the metric actually *is* the identity. Whitening provides these coordinates.

4.2 What goes wrong without whitening

Raw hidden states have a covariance matrix $C = \mathbb{E}[HH^\top]$ that is far from identity:

- Some dimensions have variance > 100 while others have variance < 0.01.
- Dimensions are correlated: $C_{ij} \neq 0$ for $i \neq j$.

If you compute the geometric product $a\,b = a \cdot b + a \wedge b$ in this distorted space, the inner product is dominated by high-variance dimensions and the bivector is tilted toward them. The "planes" you extract are artefacts of the coordinate system, not genuine geometric features of the model's computation.

Common Pitfall

This is the most common error in geometric analysis of neural networks: computing geometric quantities in non-orthonormal coordinates. Every angle, every rotation, every curvature measurement is wrong until you fix the frame.

4.3 Whitening as frame construction

Whitening via PCA constructs the orthonormal frame in three steps:

1. **Center**: subtract the mean $\bar{H}$.
2. **Diagonalize**: SVD of the covariance gives eigenvectors V and eigenvalues Λ.
3. **Scale**: project onto the top-k eigenvectors and divide by $\sqrt{\lambda_j}$. The result has covariance $= I_k$.

After whitening, the hidden states live in $\mathbb{R}^k$ with $g_{ij} = \delta_{ij}$. The algebra $\mathrm{Cl}(k, 0)$ is now valid.

Listing 4.1: Whitening constructs the orthonormal frame

```python
import layer_time_geometry as core

H_raw = core.extract_hidden_states(model.hf_model, model.tokenizer,
                                   text, model.device)
H_np = H_raw[1:].cpu().numpy()    # skip embedding layer
L, T, p = H_np.shape

metric = core.estimate_metric(H_np.reshape(-1, p), n_components=256)
H_w = core.whiten(H_np, metric)  # (L, T, k=256)

# Verify: covariance is identity
cov = np.cov(H_w.reshape(-1, 256).T)
print(f"Diagonal mean:     {np.diag(cov).mean():.4f}")    # ~1.0
print(f"|Off-diagonal| mean: "
      f"{np.abs(cov - np.diag(np.diag(cov))).mean():.6f}")  # ~0
```

4.4 Choosing k: the dimensionality

We project from $p = 3584$ to $k = 256$, retaining $> 99.5\%$ of the variance. This is a lossy projection, but the lost dimensions carry negligible information.

What the Data Shows

The geometric structure (rotor angles, curvature patterns) stabilizes by $k \approx 128$. Using $k = 256$ provides comfortable headroom. The coarse features (which layers have high rotation) are visible even at $k = 64$.

Try It Yourself

Notebook: Chapter 4. Compute the covariance matrix before and after whitening. Visualize both as heatmaps. The before-whitening heatmap should show strong off-diagonal structure; the after-whitening heatmap should be nearly diagonal.

Part II

Rotations and Rotors

CHAPTER 5

Rotations Without Matrices

Why This Matters

In standard linear algebra, a rotation is a $k \times k$ orthogonal matrix — an opaque grid of numbers. In GA, a rotation is a **rotor**: $R = \exp(-B\theta/2)$. The bivector B names the plane; θ is the angle. Plane and angle are explicit, not buried in eigenstructure. This chapter teaches you to construct, compose, and apply rotors.

5.1 From matrices to rotors

An orthogonal matrix $U \in \mathrm{SO}(k)$ (rotation, determinant $= +1$) encodes a rotation, but does not make the plane or angle obvious. To extract them, you must compute eigenvalues — and in $k > 3$ dimensions, there are multiple planes.

A **rotor** encodes the same rotation more transparently:

$$R = \exp\left(-\frac{B\theta}{2}\right)$$

where B is the unit bivector of the rotation plane and θ is the angle. The factor of $1/2$ is the GA convention (it arises because the rotor acts by *double* cover: the sandwich product).

5.2 The sandwich product

A rotor acts on a vector by the **sandwich product**:

$$v' = R\, v\, \widetilde{R}$$

where $\widetilde{R} = R^{-1} = \exp(+B\theta/2)$ is the **reversion** (the GA analog of transpose/inverse for orthogonal elements).

Key Idea

The sandwich product rotates v by angle θ in the plane B, leaving all components perpendicular to B unchanged. In matrix language: $v' = Uv$. In GA language: the plane is explicit.

5.3 Constructing rotors from orthogonal matrices

In practice, our backend computes orthogonal matrices via SVD. We then extract the bivector generator via the matrix logarithm:

$$U \in \mathrm{SO}(k) \xrightarrow{\log} A = \log(U) \text{ (skew-symmetric)} = B \text{ (bivector)}$$

Listing 5.1: From orthogonal matrix to rotor

```
from scipy.linalg import expm
from layer_time_ga.algebra import rotor_from_orthogonal

# Create a random rotation in R^8
rng = np.random.default_rng(42)
A = 0.3 * rng.standard_normal((8, 8))
A = 0.5 * (A - A.T)  # skew-symmetric
U = expm(A)           # orthogonal matrix

R = rotor_from_orthogonal(U)
print(f"Rotation angle: {R.angle:.4f} rad")
print(f"Bivector norm:  {R.bivector.norm:.4f}")

# Verify: rotation preserves vector norms
v = rng.standard_normal(8)
v_rotated = R.apply(v)
print(f"||v|| = {np.linalg.norm(v):.6f}")
print(f"||Rv|| = {np.linalg.norm(v_rotated):.6f}")
```

5.4 Rotor composition

Two rotations compose by multiplying their rotors:

$$R_{12} = R_1 \, R_2$$

In matrix language: $U_{12} = U_1 U_2$. But the rotor form reveals more. If R_1 and R_2 act in the *same* plane, the composition simply adds the angles. If they act in *different* planes, the composition mixes the planes — and the resulting bivector reveals the combined plane.

Listing 5.2: Composing and inverting rotors

```
from layer_time_ga.algebra import rotor_compose, rotor_inverse

R2 = rotor_from_orthogonal(expm(0.15 * (rng.standard_normal((8,8))
```

```
    - rng.standard_normal((8,8)).T) / 2))

R12 = rotor_compose(R, R2)
print(f"Composed angle: {R12.angle:.4f}")

R_inv = rotor_inverse(R)
R_id = rotor_compose(R, R_inv)
print(f"R * R^-1 deviation: {R_id.deviation_from_identity():.2e}")
```

In the Transformer

Every layer transition in a transformer contains a rotation. The rotor at layer l has a bivector $\mathcal{B}^{(l)}$ encoding which planes the layer rotates in. The collection of rotors across all layers is the **rotor field** — a discrete gauge connection on the layer axis. Chapter 6 shows how to extract it from real data.

Try It Yourself

Notebook: Chapter 5.

1. Create a rotor for a 45-degree rotation in the $e_1 \wedge e_2$ plane. Apply it to e_1. Verify you get $(\cos 45^\circ)\, e_1 + (\sin 45^\circ)\, e_2$.
2. Compose two rotors in the same plane. Verify the angles add.
3. Compose two rotors in perpendicular planes. Extract the principal planes of the composed rotor. What do you see?

CHAPTER 6

What a Layer Actually Does

Why This Matters

We now apply our GA toolkit to real transformer data. Each layer transition decomposes into a rotor (rotation, grade-2) and a metric factor (stretching, grade-0). This is the **versor decomposition**, the GA analog of the polar decomposition. We build the rotor field and see how information rotates and stretches across the entire network.

6.1 The versor decomposition

Each layer transition operator $T^{(l)}$ (the linear map that best approximates the change $H^{(l+1)} \approx T^{(l)} H^{(l)}$) factors as:

$$T^{(l)} = \mathcal{R}^{(l)} \mathcal{P}^{(l)}$$

- $\mathcal{R}^{(l)}$ is a **rotor** (orthogonal, $\det = +1$): it rotates the hidden state into a new orientation.
- $\mathcal{P}^{(l)}$ is a **metric factor** (symmetric, positive-definite): it stretches some directions and compresses others.

In matrix language, this is the polar decomposition $T = UP$. In GA language, it separates **grade-2 content** (the rotation plane and angle) from **grade-0 content** (the eigenvalues of the stretch).

6.2 Extracting the rotor field

Listing 6.1: Extracting the rotor field from real transformer data

```python
import ltg_ga
from layer_time_ga.decomposition import extract_rotor_field

model = ltg_ga.load_model("Qwen/Qwen2.5-7B")
result = ltg_ga.analyse("What is 2 + 3?", model=model)

```

```python
rf = result.rotor_field  # LayerRotorField

print(f"{'Layer':>6} {'Angle':>8} {'kappa':>8} {'erank':>8}")
print("-" * 36)
for i, vd in enumerate(rf.decompositions):
    print(f"{i:6d} {vd.rotor.angle:8.4f} "
          f"{vd.condition_number:8.1f} "
          f"{vd.effective_rank:8.1f}")
```

6.3 What the rotor field tells you

The rotor field has three main signatures:

What the Data Shows

1. Rotation angle profile. The angle $\theta^{(l)}$ measures how much the hidden states rotate at each layer. Typically: moderate rotation in early layers, a dip in the middle, and a peak in the final layers as the model converges to a prediction.

What the Data Shows

2. Condition number profile. The condition number $\kappa^{(l)}$ of the metric factor $\mathcal{P}^{(l)}$ measures how selective the layer is. Late layers have $\kappa \gg 1$: they strongly amplify a few directions and suppress the rest.

What the Data Shows

3. Effective rank profile. The effective rank $\text{erank}^{(l)}$ measures how many directions are actively used. It decreases from early to late layers: the representation concentrates into a lower-dimensional subspace.

6.4 Grade-0 vs grade-2: which dominates?

For each layer, compare:

$$\text{Grade-0 deviation:} \quad \|\mathcal{P}^{(l)} - I\|_F \quad \text{(how far from identity stretch)} \tag{6.1}$$

$$\text{Grade-2 content:} \quad \|\mathcal{B}^{(l)}\|_F \quad \text{(how much rotation)} \tag{6.2}$$

What the Data Shows

Grade-0 dominates grade-2 in late layers. The metric deformation (stretching) is 5–10× larger than the rotation in the final layers. This means late layers primarily *select directions* (amplify/compress) rather than *reorient* (rotate).

In the Transformer

This has a profound implication for controlling model behavior. Modifying eigenvalues of $\mathcal{P}^{(l)}$ (grade-0 intervention) changes D_{total} by ∼48%. Modifying the rotor (grade-2 intervention) changes it by only ∼9%. The model's output is controlled by *what it amplifies*, not *where it points*.

Try It Yourself

Notebook: Chapter 6. Plot the grade-0 and grade-2 content side by side for each layer. At which layer does grade-0 begin to dominate? Compare two prompts: does the crossover point change?

CHAPTER 7

Reading the Planes

Why This Matters

We can now extract bivectors from layer transitions. But a bivector in $\mathbb{R}^{256}$ has 32,640 components. How do we make sense of it? This chapter teaches you to decompose bivectors into their principal planes and track how these planes evolve across the network. This is the unique diagnostic capability that GA provides.

7.1 Principal plane extraction

The bivector $\mathcal{B}^{(l)}$ at layer l is a $k \times k$ skew-symmetric matrix. Its eigendecomposition gives conjugate pairs $\pm i\sigma_j$, where:

- σ_j is the **rotation angle** in plane j.
- The corresponding eigenvectors span the **plane**.

Ordering by σ_j gives the principal planes. Typically, the top 3–5 planes carry $> 80\%$ of the total rotation.

Listing 7.1: Principal planes of a layer's bivector

```
# Inspect layer 20's rotation planes
vd = rf.decompositions[20]
planes = vd.bivector.principal_planes(n_planes=5)
print(f"Layer 20: total angle = {vd.rotor.angle:.4f}")
for j, p in enumerate(planes):
    print(f"  Plane {j}: angle = {p['angle']:.4f}, "
          f"weight = {p['weight']:.4f}")
```

7.2 Plane evolution across layers

Extract the top plane at each layer. Track the plane's orientation (the eigenvectors) and its weight (the fraction of total rotation).

Questions this answers:

1. Do early and late layers rotate in the same planes? (Usually no: the dominant plane shifts.)
2. Is there a "phase transition" where the rotation structure changes qualitatively?
3. Do different prompts produce different rotation planes? (Often yes, in the final layers.)

GA Concept

This is the core GA diagnostic. Everything we build from here — holonomy, commutators, capacity — preserves the bivector and its plane information. The first-edition scalar summaries ($\|U - I\|_F$, $\|[A_i, A_j]\|_F$, $\|\Omega - I\|_F$) are the *norms* of these bivectors. GA keeps the full directional information.

7.3 Plane similarity between layers

To compare rotation planes at layers l and l', compute the cosine similarity between their bivector matrices:

$$\mathrm{sim}(l, l') = \frac{\langle \mathcal{B}^{(l)}, \mathcal{B}^{(l')} \rangle}{\|\mathcal{B}^{(l)}\| \, \|\mathcal{B}^{(l')}\|} = \frac{\mathrm{tr}(\mathcal{B}^{(l)\top} \mathcal{B}^{(l')})}{\|\mathcal{B}^{(l)}\|_F \, \|\mathcal{B}^{(l')}\|_F}$$

High similarity means the two layers rotate in similar planes. Low similarity means they perform qualitatively different geometric operations.

Try It Yourself

Notebook: Chapter 7.

1. Compute the bivector similarity matrix (all pairs of layers). Plot as a heatmap. Do you see block structure?
2. For the top principal plane at each layer, track its evolution as a stacked area chart of weights across layers.
3. Compare the plane similarity matrix for an arithmetic prompt vs. a reasoning prompt. Do they use different planes?

CHAPTER 8

The Eigenvalue Story

Why This Matters

The metric factor $\mathcal{P}^{(l)}$ is the grade-0 part of the versor decomposition. Its eigenvalues determine which directions are amplified and which are suppressed. This chapter teaches you to read the eigenvalue spectrum and understand why scalars (grade-0) control the model's behavior more than rotations (grade-2).

8.1 Eigenvalue spectra

The metric factor $\mathcal{P}^{(l)}$ is a symmetric positive-definite matrix with eigenvalues $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k > 0$.

- $\sigma_i = 1$ for all i: the metric factor is the identity; the layer only rotates (no stretching).
- $\sigma_1 \gg \sigma_k$: the layer strongly amplifies some directions and suppresses others.
- $\kappa = \sigma_1/\sigma_k$: the **condition number**.
- $\text{erank} = \exp H(\hat{\sigma})$: the **effective rank**, where $\hat{\sigma}_i = \sigma_i / \sum \sigma_j$.

8.2 Why grade-0 dominates

Key Idea

Rotation (grade-2) preserves norms by definition: $\|Rv\| = \|v\|$. Stretching (grade-0) changes norms: $\|Pv\| \neq \|v\|$ in general.

During backpropagation, gradient magnitudes are controlled by the eigenvalues of P, not by the rotation. This is why modifying eigenvalues (grade-0) has a 48% effect on dependency while modifying the rotor (grade-2) has only a 9% effect.

Under the Hood

For a layer with versor decomposition $T = RP$, the Jacobian of the layer w.r.t. the input is T itself. During backpropagation, the gradient is multiplied by $T^\top = P^\top R^\top = PR^\top$. The rotation $R^\top$ preserves the gradient's magnitude; the stretching P amplifies or suppresses it. Gradient flow is controlled by the singular values of P.

8.3 Reading the spectrum

Listing 8.1: Eigenvalue spectra of three layers

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, idx, label in zip(axes,
    [0, len(rf.decompositions)//2, -1],
    ['Early', 'Middle', 'Late']):
    vd = rf.decompositions[idx]
    sv = np.sort(vd.singular_values)[::-1]
    ax.bar(range(len(sv)), sv, width=1.0, color='#1D6A6A')
    ax.axhline(y=1.0, color='red', linestyle='--', alpha=0.5,
               label='identity')
    ax.set_title(f'{label} (layer {vd.layer_index},'
                 f' kappa={vd.condition_number:.0f})')
    ax.set_xlabel('Direction')
    ax.set_ylabel('Eigenvalue')
    ax.legend()
plt.tight_layout()
```

What the Data Shows

Late layers are highly selective. The eigenvalue spectrum becomes increasingly peaked toward the output: a handful of directions carry most of the signal, while the rest are suppressed. This is the metric factor implementing the model's "decision" about which information to propagate to the final prediction.

Try It Yourself

Notebook: Chapter 8.

1. Plot the condition number $\kappa^{(l)}$ and effective rank $\text{erank}^{(l)}$ across layers. Where does selectivity peak?
2. For the most selective layer, identify the top-3 eigenvectors. These are the directions the model amplifies most.
3. Compare eigenvalue spectra for two prompts. Do the same directions get amplified?

Part III

Curvature and Commutators

CHAPTER 9

When Order Matters

Why This Matters

Two rotations applied in different orders can give different results. The **commutator** measures this failure to commute. In GA, the commutator of two bivectors is itself a bivector — telling you not just *how much* the rotations fail to commute, but *in which plane* the disagreement lives. This is the Lie algebra of the rotation group, and it is the algebraic backbone of curvature and capacity.

9.1 The commutator as a bivector

For bivectors B_1, B_2 (skew-symmetric matrices), the **commutator** or **Lie bracket** is:

$$[B_1, B_2] = B_1 B_2 - B_2 B_1$$

Key properties:

1. The result is a skew-symmetric matrix, hence a **bivector**.
2. **Antisymmetry**: $[B_1, B_2] = -[B_2, B_1]$.
3. **Self-commutator is zero**: $[B, B] = 0$.
4. **Jacobi identity**: $[A, [B, C]] + [B, [C, A]] + [C, [A, B]] = 0$.

GA Concept

The Lie algebra $\mathfrak{so}(k)$.

The set of all bivectors (skew-symmetric $k \times k$ matrices) forms a **Lie algebra** under the commutator bracket. This is $\mathfrak{so}(k)$, the infinitesimal version of the rotation group. The commutator tells you how two infinitesimal rotations interact. If $[B_1, B_2] = 0$, they commute (order does not matter). If not, the commutator bivector names the plane of disagreement.

9.2 Computing commutators

Listing 9.1: Commutator of two bivectors

```
from layer_time_ga.algebra import commutator_bivector

# Bivectors from two adjacent layer transitions
B1 = rf.bivectors[10]  # layer 10
B2 = rf.bivectors[11]  # layer 11

comm = commutator_bivector(B1, B2)
print(f"||[B1, B2]||_F = {comm.norm:.6f}")
print(f"Antisymmetric: {np.allclose(comm.matrix, -comm.matrix.T)}")

# The commutator's principal planes
comm_planes = comm.principal_planes(n_planes=2)
for i, p in enumerate(comm_planes):
    print(f"  Plane {i}: angle = {p['angle']:.4f}")
print("This directional info is invisible to ||[A_i,A_j]||_F!")
```

9.3 The commutator field

For all pairs of layers, compute $\|[\mathcal{B}^{(i)}, \mathcal{B}^{(j)}]\|_F$. This gives a symmetric matrix (the **commutator field**) that shows which layer pairs interact non-trivially.

Listing 9.2: The commutator heatmap

```
from layer_time_ga.curvature import commutator_field

comm_norms = commutator_field(rf.bivectors)
# comm_norms[i, j] = ||[B_i, B_j]||_F

plt.imshow(comm_norms, cmap='inferno', origin='lower')
plt.colorbar(label='$||[B_i, B_j]||_F$')
plt.xlabel('Layer $j$'); plt.ylabel('Layer $i$')
plt.title('Commutator Norm Heatmap')
```

What the Data Shows

The commutator heatmap often shows **block structure**: groups of layers that strongly interact (high commutator norms between them) and groups that are nearly commutative (low norms). This block structure varies with prompt type and model architecture.

Try It Yourself

Notebook: Chapter 9.

1. Verify the Jacobi identity for three random bivectors from the rotor field.
2. For the top-3 most non-commuting layer pairs, extract the commutator's principal planes. Do they share a common plane?
3. Plot commutator norms for adjacent pairs only: $\|[\mathcal{B}^{(l)}, \mathcal{B}^{(l+1)}]\|_F$. Where is non-commutativity strongest?

CHAPTER 10

Walking in Circles

Why This Matters

Holonomy is the most elegant concept in differential geometry: transport a vector around a closed loop and see if it comes back unchanged. If not, the space is curved. In our setting, the "space" is the layer–time grid, the "transport" is done by rotors, and the holonomy rotor's bivector points in the *direction* of curvature.

10.1 Parallel transport on the grid

The layer–time grid has two axes: layer (l) and token (t). At each grid point, there are two ways to move: up one layer, or right one token. Each move applies a local rotor.

To test for curvature at a **plaquette** (l, t) (the unit square from (l, t) to $(l+1, t+1)$), compose rotors around the loop:

$$R_{\text{loop}} = R_{\rightarrow}^{(l+1)} \, (R_{\uparrow}^{(t+1)})^{-1} \, (R_{\rightarrow}^{(l)})^{-1} \, R_{\uparrow}^{(t)}$$

Analogy

Imagine walking along the surface of the Earth. Start at the equator, walk north to the pole, turn right 90 degrees, walk south to the equator, turn right 90 degrees, walk back to your start. You end up facing a different direction than when you started — even though you only ever walked "straight" and turned at corners. The mismatch is holonomy; it exists because the Earth's surface is curved.

The same thing happens on the layer–time grid. If the model applies different transformations depending on whether you go "layer first, then token" vs. "token first, then layer," the grid is curved at that plaquette.

10.2 The holonomy rotor

If $R_{\text{loop}} = I$ (identity rotor), the plaquette is flat. Otherwise:

- $\|R_{\text{loop}} - I\|_F$ = **scalar curvature** (a number).
- $\log(R_{\text{loop}})$ = **curvature bivector** (an oriented plane — *which* plane is curved).

Key Idea

Scalar curvature tells you *how much*. The holonomy bivector tells you *which direction*. Two plaquettes can have the same scalar curvature but curve in different planes. The bivector distinguishes them.

10.3 Computing holonomy

Listing 10.1: Holonomy at a single plaquette

```
from layer_time_ga.curvature import holonomy_rotor, holonomy_scalar_map

# Single plaquette
hr = holonomy_rotor(result.H_whitened, l=20, t=2)
print(f"Scalar curvature: {hr.scalar_curvature:.6f}")
print(f"Bivector norm:    {hr.bivector_norm:.6f}")
if hr.principal_plane:
    print(f"Curvature plane:  angle={hr.principal_plane['angle']:.4f}")

# Full scalar map
holo_map = holonomy_scalar_map(result.H_whitened)
print(f"Shape: {holo_map.shape}")  # (L-1, T-1)
```

10.4 Curvature patterns in transformers

What the Data Shows

Curvature concentrates in the final layers. The last 3–5 layers contain 60–80% of the total curvature. This is where the model's computation is most non-trivial: the layer and token axes interact strongly as the model converges to a prediction.

What the Data Shows

Curvature planes vary across tokens. At a fixed layer, the holonomy bivector can point in different directions for different tokens. This means the model processes each token through a *different* curved region of representation space — a form of token-specific computation.

Try It Yourself

Notebook: Chapter 10.

1. Plot the holonomy scalar map as a heatmap. Where is curvature highest?
2. For the 5 highest-curvature plaquettes, extract the holonomy bivector's principal plane. Do they share a common curvature direction?
3. Compare holonomy maps for two prompts. Do the curvature patterns differ in location, magnitude, or direction?

CHAPTER 11

How Much Computation?

Why This Matters

Capacity measures the total "computational work" performed by the network. In GA language, this is the accumulated non-commutativity of the bivector field. We can decompose capacity by plane, by layer, and by dependency weight — giving a complete picture of where and how the model's computation is structured.

11.1 Accumulated non-commutativity

The **accumulated capacity** sums commutator norms across adjacent layers:

$$C_{\text{acc}} = \sum_{l=1}^{L-1} \|[\mathcal{B}^{(l)}, \mathcal{B}^{(l+1)}]\|_F$$

Each term measures how much layers l and $l+1$ fail to commute. If all layers commuted perfectly (a flat connection), $C_{\text{acc}} = 0$. Higher values mean more complex geometric transformation.

11.2 Dependency weighting

Not all non-commutativity matters for the output. Weighting by gradient-based dependency D_l gives the **effective capacity**:

$$C_{\text{eff}} = \sum_{l} D_l \cdot \|[\mathcal{B}^{(l)}, \mathcal{B}^{(l+1)}]\|_F$$

This focuses on the non-commutativity that actually drives the model's prediction.

11.3 Plane decomposition of capacity

The commutator bivectors $[\mathcal{B}^{(l)}, \mathcal{B}^{(l+1)}]$ can be aggregated and decomposed into principal planes:

Listing 11.1: Capacity with plane decomposition

```
from layer_time_ga.curvature import commutator_plane_decomposition

planes = commutator_plane_decomposition(rf.bivectors, n_planes=5)
print(f"Total commutator norm: {planes['total_norm']:.2f}")
for i, p in enumerate(planes['principal_planes']):
    pct = p['weight'] / planes['total_norm'] * 100
    print(f"  Plane {i}: weight = {p['weight']:.4f} ({pct:.1f}%)")
```

If most capacity is concentrated in a few planes, the model's computation is "low-rank" in the bivector sense: it uses only a few planes of rotation. If capacity is spread, the computation is high-dimensional.

GA Concept

The Jacobi identity as a consistency check.

The Jacobi identity $[A, [B, C]] + [B, [C, A]] + [C, [A, B]] = 0$ must hold for any three bivectors. This is a non-trivial constraint on the commutator field. If your numerical computation violates the Jacobi identity significantly, something is wrong with the data or the decomposition.

What the Data Shows

Distilled models concentrate capacity in fewer planes. The DeepSeek-R1-Distill model focuses its non-commutativity in fewer principal planes than the base Qwen model of the same size. Distillation appears to "clean up" the rotation structure by removing non-commutativity in planes that do not contribute to performance.

Try It Yourself

Notebook: Chapter 11.

1. Compute C_{acc} for prompts of varying complexity. Does harder = higher capacity?
2. Verify the Jacobi identity for three bivectors from the rotor field. How close to zero is the sum?
3. Decompose the commutator structure into principal planes. How many planes capture 90% of the total norm?

Part IV

The Full Picture

CHAPTER 12

Which Planes Matter?

Why This Matters

We now have a rich geometric picture: bivectors, rotors, holonomy, commutators, eigenvalue spectra. But which of these geometric features actually matter for the model's output? **Dependency density** answers this question by measuring gradient sensitivity at each layer. Combined with the bivector structure, it tells us which *planes of rotation* drive the prediction.

12.1 Dependency density

The dependency at layer l is the Frobenius norm of the gradient of the output logits with respect to the hidden state:

$$D_l = \left\| \frac{\partial s}{\partial H^{(l)}} \right\|_F$$

This is a scalar per layer. It does not depend on GA — it is a standard gradient computation.

Key summaries:

- $D_{\text{total}} = \sum_l D_l$: total sensitivity.
- $H(D)$: entropy of the normalized profile. Higher = more distributed computation.
- Horizon-90: the layer where cumulative dependency reaches 90%.

12.2 Effective work: dependency $\times$ rotation

The product $D_l \cdot \theta^{(l)}$ is the **effective geometric work** at layer l. It combines "how much the model cares" with "how much the layer rotates."

What the Data Shows

Peak effective work does not coincide with peak rotation. The most geometrically active layer (highest θ) is often *not* the most important layer (highest D_l). Effective work identifies the layers that are both active and important.

12.3 Dependency-weighted bivectors

For each high-effective-work layer, extract the principal rotation plane. This gives a short, precise description:

"The model's prediction depends primarily on rotations in planes $\hat{B}_1, \hat{B}_2, \hat{B}_3$ at layers l_1, l_2, l_3."

Listing 12.1: Identifying the functionally relevant planes

```
# Effective work per layer
eff_work = result.dependency_profile * result.angles[:len(result.dependency_profile)
    ]

# Top-3 layers by effective work
top_layers = np.argsort(eff_work)[-3:][::-1]

for l in top_layers:
    vd = rf.decompositions[l]
    plane = vd.bivector.principal_planes(n_planes=1)[0]
    print(f"Layer {l}: D={result.dependency_profile[l]:.4f}, "
          f"theta={result.angles[l]:.4f}, "
          f"plane_angle={plane['angle']:.4f}")
```

In the Transformer

This is the ultimate diagnostic: a list of specific layers and specific planes that drive the model's output. If you want to steer the model's behavior, intervene at these layers, in these planes. If you want to understand a failure, look at what these planes are doing.

Try It Yourself

Notebook: Chapter 12.

1. Plot dependency and rotor angles on the same figure (twin axes). Where do they overlap?
2. Compute effective work for three prompts of different types. Do the top layers differ?
3. For the top-3 effective-work layers, are the dominant rotation planes similar across prompts, or prompt-specific?

CHAPTER 13

Diagnosing and Steering

Why This Matters

This chapter applies everything you have learned to three practical problems: detecting when a model ignores context, diagnosing hallucinations, and steering model behavior via plane-specific interventions. Each application uses the full GA toolkit: bivectors, holonomy, commutators, dependency.

13.1 Application 1: Does the model use context?

The method

Compare the GA profile of a prompt with and without relevant context. If the model uses context, its geometric profile should change — different curvature planes, different dominant rotation planes.

The GA diagnostic

Listing 13.1: Context-ignoring detection via curvature planes

```
r_with = ltg_ga.analyse(prompt_with_context, model=model)
r_without = ltg_ga.analyse(prompt_without_context, model=model)

# Compare curvature planes at the peak curvature layer
from layer_time_ga.curvature import holonomy_rotor

peak = r_with.curvature_by_layer.argmax()
hr1 = holonomy_rotor(r_with.H_whitened, l=peak, t=0)
hr2 = holonomy_rotor(r_without.H_whitened, l=peak, t=0)

# Cosine similarity between curvature bivectors
cos_sim = np.sum(hr1.bivector.matrix * hr2.bivector.matrix)
cos_sim /= (hr1.bivector.norm * hr2.bivector.norm + 1e-12)
```

```
# Low similarity => model IS using context
```

What the Data Shows

Curvature plane shift is a stronger indicator than magnitude change. Scalar curvature may change by only 10–20%, but the *direction* of curvature can shift substantially, indicating qualitatively different computation.

13.2 Application 2: Hallucination signatures

Hallucinations are associated with "busy but useless" layers: high geometric activity (curvature, rotation) but low dependency. GA adds plane-level diagnostics:

- **Consistent curvature plane**: the model is "confidently wrong" — it performs systematic but misdirected computation.
- **Random curvature planes**: the model is "lost" — curvature directions vary across tokens and layers without coherent structure.

13.3 Application 3: Plane-specific steering

Key Idea

Instead of modifying all directions at a target layer, intervene only in the **principal rotation plane**:

1. Identify target layers by effective work $D_l \cdot \theta_l$.
2. Extract the dominant bivector plane at each target layer.
3. Modify the eigenvalues of $\mathcal{P}^{(l)}$ only in the 2D subspace of that plane (amplify or suppress).

This is a rank-2 intervention in a k-dimensional space — maximally targeted.

Try It Yourself

Notebook: Chapter 13.

1. Run the context-ignoring detection on a factual question with and without a relevant paragraph. Does the curvature plane shift?
2. Generate the GA summary plot for a prompt you suspect will produce a hallucination. Look for the "busy but useless" layers.
3. Identify the top-2 steering targets (layer + plane) for a given prompt.

CHAPTER 14

What Geometric Algebra Gave Us

14.1 The journey

You started with vectors and the dot product. You now have:

Chapter	GA concept	Transformer insight
1–2	Vectors, geometric product	Hidden states encode both alignment and plane information
3	Bivectors	Layer rotations have principal planes, not just magnitudes
4	Orthonormal frames	Whitening establishes the metric for valid GA
5	Rotors	Layer transitions are rotations with explicit planes
6	Versor decomposition	Grade-0 (stretch) vs. grade-2 (rotation)
7	Principal planes	Rotation plane evolution across the network
8	Eigenvalues	Grade-0 dominates control; late layers are highly selective
9	Commutators	Non-commutativity lives in specific planes
10	Holonomy	Curvature has a direction, not just a magnitude
11	Capacity	Decompose computational complexity by plane
12	Dependency + bivectors	Identify the planes that matter
13	Applications	Diagnose and steer via plane-specific interventions

14.2 What was invisible before

Scalar summaries ($\|U - I\|_F$, $\|[A_i, A_j]\|_F$, $\|\Omega - I\|_F$) tell you *how much*. GA tells you *how much* and *in which direction*. Specifically:

1. Which planes each layer rotates in.
2. Which planes the curvature acts in.
3. Which planes different layers fail to commute in.
4. Which planes the model's output depends on.

None of this information is accessible from norms alone. It requires the full bivector.

14.3 Where to go next

1. **Higher grades.** We used only grades 0 and 2. Do trivectors (grade 3) carry meaningful information about three-layer interactions?
2. **Conformal GA.** The conformal model $\mathrm{Cl}(k+1,1)$ adds translations to the algebra. Could this unify residual connections (translations) with rotations (rotors)?
3. **GA in training dynamics.** How does the bivector field evolve during pre-training? Do specific planes emerge at specific training stages?
4. **Multimodal models.** When vision and language tokens interact, do they occupy different rotation planes?
5. **Projective GA.** $\mathrm{Cl}(k,0,1)$ naturally handles the affine structure of residual connections.

14.4 Further reading

- Doran and Lasenby, *Geometric Algebra for Physicists* (Cambridge, 2003). The most accessible GA introduction.
- Dorst, Fontijne, and Mann, *Geometric Algebra for Computer Science* (Morgan Kaufmann, 2007). Practical and code-oriented.
- Lounesto, *Clifford Algebras and Spinors* (Cambridge, 2001). The mathematical reference.
- Sudjianto and Zhang, *Layer–Time Geometry: How Language Models Process Information* (2026). The first-edition linear algebra treatment.
- Fang, Li, and Sudjianto, *Design and Modeling for Computer Experiments* (Chapman & Hall/CRC, 2006). Background on DOE methodology.

APPENDIX A

Transformers: A Primer

This appendix provides the minimum transformer background needed for the book.

A.1 The architecture

A transformer consists of L identical layers. Each layer has:

1. **Multi-head self-attention**: each token computes a weighted average of all tokens' representations.
2. **Feed-forward network**: a two-layer MLP applied to each token independently.
3. **Residual connections**: the output is added to the input: $H^{(l+1)} = H^{(l)} + \Delta H^{(l)}$.
4. **Layer normalization**: stabilizes the scale of representations.

The hidden state $H(l, t) \in \mathbb{R}^p$ at layer l and token position t is the model's internal representation.

A.2 Autoregressive prediction

GPT-style models predict the next token given all preceding tokens: $P(x_t \mid x_{<t})$. The final hidden state $H(L, t)$ is projected to vocabulary logits, and the highest-probability token is selected.

A.3 The models we study

- **Qwen-2.5-7B**: 28 layers, hidden dimension 3584, 7 billion parameters.
- **Qwen-2.5-32B**: 64 layers, hidden dimension 5120, 32 billion parameters.
- **DeepSeek-R1-Distill-Qwen-7B**: same architecture as Qwen-7B, distilled from a larger reasoning model.

APPENDIX B

GA from Scratch

This appendix provides the formal mathematical construction of the Clifford algebra for readers who want the full picture.

B.1 The Clifford algebra $\mathrm{Cl}(k, 0)$

Given a k-dimensional real vector space V with positive-definite inner product, the Clifford algebra is the associative algebra generated by V under:

$$v^2 = v \cdot v = \|v\|^2 \quad \text{for all } v \in V.$$

For orthonormal basis vectors e_i:

$$e_i^2 = 1, \qquad e_i e_j = -e_j e_i \quad (i \neq j).$$

The algebra has dimension 2^k, graded by degree: $\mathrm{Cl}(k, 0) = \bigoplus_{r=0}^{k} \bigwedge^r V$.

B.2 Blades and multivectors

A r**-blade** $B_r = v_1 \wedge \cdots \wedge v_r$ represents an oriented r-dimensional subspace. A general grade-r element is a linear combination of r-blades. A **multivector** is a sum of elements of different grades.

B.3 The spin group

The set of rotors R with $R\widetilde{R} = 1$ forms the **spin group** $\mathrm{Spin}(k)$, which double-covers $\mathrm{SO}(k)$. Every rotation can be written as:

$$R = \exp\left(-\frac{B\theta}{2}\right)$$

for a unit bivector B and angle θ.

B.4 The Lie algebra

Bivectors form a Lie algebra under the commutator: $[B_1, B_2] = B_1 B_2 - B_2 B_1$. This is isomorphic to $\mathfrak{so}(k)$. The dimension is $k(k-1)/2$.

B.5 Duality and the pseudoscalar

The pseudoscalar $I = e_1 e_2 \cdots e_k$ maps grade-r to grade-$(k-r)$. In $\mathbb{R}^3$, this relates bivectors to vectors (the cross product). In higher dimensions, bivectors are irreducible and more general than vectors.

APPENDIX C

The Matrix–Multivector Bridge

Matrix operation	**GA equivalent**	**What GA adds**
Dot product $a^\top b$	Inner product $a \cdot b$	Same computation
$ab^\top - ba^\top$ (skew)	Outer product $a \wedge b$	Named as a bivector (oriented plane)
Skew-symmetric A	Bivector B	Principal plane decomposition
Orthogonal $U \in \mathrm{SO}(k)$	Rotor $R = e^{-B/2}$	Explicit plane + angle
Polar decomposition $T = UP$	Versor decomposition $T = RP$	Grade-2 $\times$ grade-0 separation
$\|U - I\|_F$	Rotor angle θ	Plus: which plane
$A_i A_j - A_j A_i$	$[B_i, B_j]$	Plus: decompose into planes
$\|\Omega - I\|_F$ (curvature)	Holonomy rotor	Plus: curvature direction
Eigenvalues of P	Grade-0 content	Natural grade hierarchy

APPENDIX D

Code Reference

D.1 Student API: `ltg_ga`

```python
import ltg_ga

model = ltg_ga.load_model("Qwen/Qwen2.5-7B")
result = ltg_ga.analyse("prompt", model=model)
result.summary()
result.plot_ga_summary(save_path="summary.png")

# Compare multiple prompts
ltg_ga.compare([r1, r2, r3], save_path="compare.png")

# Capacity analysis
cap = ltg_ga.capacity("prompt", model=model)
```

D.2 Core algebra: `layer_time_ga.algebra`

```python
from layer_time_ga.algebra import (
    Bivector, Rotor,
    bivector_from_skew, rotor_from_orthogonal,
    rotor_compose, rotor_inverse,
    commutator_bivector, grade_decomposition,
    geometric_product_vectors,
)
```

D.3 Decomposition: `layer_time_ga.decomposition`

```python
from layer_time_ga.decomposition import (
    VersorDecomposition, LayerRotorField,
```

```
    versor_decompose, extract_rotor_field,
    extract_bivector_field, extract_metric_field,
)
```

D.4 Curvature: `layer`$_t$*ime_ga.curvature*

```
from layer_time_ga.curvature import (
    holonomy_rotor, holonomy_field, holonomy_scalar_map,
    commutator_field, commutator_bivectors,
    commutator_plane_decomposition,
)
```

D.5 Capacity: `layer_time_ga.capacity`

```
from layer_time_ga.capacity import (
    ga_capacity_profile, GACapacityProfile,
)
```

Glossary

Term	Definition
Bivector	A grade-2 element representing an oriented plane; stored as a skew-symmetric matrix.
Blade	A bivector (or higher-grade element) that can be written as a single outer product $a \wedge b$; represents one plane.
Clifford algebra	The algebra $\mathrm{Cl}(k, 0)$ generated by vectors under $v^2 = \|v\|^2$. Contains all grades from 0 to k.
Commutator	$[B_1, B_2] = B_1 B_2 - B_2 B_1$; a bivector measuring non-commutativity.
Curvature bivector	$\log(R_{\text{loop}})$: the plane in which space is curved at a grid point.
Geometric product	$ab = a \cdot b + a \wedge b$; the fundamental operation of GA.
Grade	The "type" of an algebraic element: 0 = scalar, 1 = vector, 2 = bivector.
Holonomy	The loop rotor from composing rotors around a closed path; measures curvature.
Metric factor	The symmetric positive-definite matrix P from $T = RP$; encodes grade-0 stretching.
Orthonormal frame	Basis vectors with $e_i \cdot e_j = \delta_{ij}$; required for valid GA operations.
Principal plane	A simple-bivector component of a compound bivector; extracted via eigendecomposition.
Reversion	The GA analog of transpose: $\widetilde{R} = R^{-1}$ for rotors.
Rotor	An even-grade element $R = \exp(-B\theta/2)$ representing a rotation in plane B by angle θ.
Sandwich product	$v' = Rv\widetilde{R}$; how a rotor acts on a vector.
Versor decomposition	$T = RP$: separating rotation (grade-2) from stretching (grade-0).
Whitening	PCA-based dimensionality reduction that constructs an orthonormal frame.